

CLASS NOTE: Creating a Simple Node.js Server (with JSON & Express Introduction)

Class Objectives

By the end of this class, students will:

1. Understand what a **server** is and how it works in **Node.js**.
2. Create a simple **HTTP server** that sends **JSON responses**.
3. Handle **different routes** using **conditional logic**.
4. Learn **why Express.js** is used and **how it improves Node.js development**.

Module 1: Understanding What a Server Is

Keyword Highlights

- **Server** → A program that listens for **client requests** and returns **responses**.
- **Client** → The program (browser, mobile app, or Postman) that sends the request.
- **Port** → A virtual channel for communication (e.g., 3000 , 8080).
- **Request (req)** → The client's data sent to the server.
- **Response (res)** → The server's reply back to the client.

Simple Definition

A **server** is like a restaurant waiter — it **receives your order (request)** and **brings back your meal (response)**.

Module 2: Setting Up Node.js Environment

Setup Steps

1. Install Node.js → <https://nodejs.org>
2. Create a new folder

```
mkdir json-server  
cd json-server
```

3. Initialize project

```
npm init -y
```

4. Create your main file

```
touch server.js
```

Module 3: Creating a Basic Node.js JSON Server

Keyword Highlights

- `require()` → Imports built-in modules.
- `http` → Core Node.js module for creating servers.
- `JSON.stringify()` → Converts JavaScript objects to JSON strings.
- `Content-Type` → Specifies the type of data sent (e.g., `application/json`).

Example Code

`server.js`

```
// Import the http module
const http = require("http");

// Create the server
const server = http.createServer((req, res) => {
  res.setHeader("Content-Type", "application/json");

  if (req.url === "/") {
    res.statusCode = 200;
    res.end(JSON.stringify({ message: "Welcome to our Node.js JSON Server!" }));
  } else if (req.url === "/students") {
    const students = [
      { id: 1, name: "John Doe", course: "Node.js" },
      { id: 2, name: "Jane Smith", course: "JavaScript" },
    ];
    res.statusCode = 200;
    res.end(JSON.stringify(students));
  } else if (req.url === "/info") {
    const info = { appName: "Simple JSON Server", version: "1.0.0" };
    res.statusCode = 200;
    res.end(JSON.stringify(info));
  } else {
    res.statusCode = 404;
    res.end(JSON.stringify({ error: "Route not found" }));
  }
});

// Listen on port 3000
server.listen(3000, () => {
  console.log("Server is running at http://localhost:3000");
});
```

Code Explanation

Keyword/Method	Meaning	Example
<code>require('http')</code>	Loads the built-in HTTP module	<code>const http = require('http');</code>
<code>http.createServer()</code>	Creates a new web server	<code>(req, res) => { ... }</code>
<code>res.setHeader()</code>	Sets the response type	<code>'Content-Type': 'application/json'</code>

Keyword/Method	Meaning	Example
<code>res.statusCode</code>	Defines HTTP status (200 OK, 404 Not Found, etc.)	<code>res.statusCode = 200</code>
<code>res.end()</code>	Ends the response and sends data back	<code>res.end(JSON.stringify(obj))</code>
<code>JSON.stringify()</code>	Converts an object to JSON	<code>{ name: 'Lee' } → '{"name": "Lee"}'</code>

Module 4: Testing the Server

Steps

1. Run your server:

```
node server.js
```

2. Open a browser or Postman:

- `http://localhost:3000/` → Welcome message
- `http://localhost:3000/students` → Returns a list of students
- `http://localhost:3000/info` → Shows app details
- `http://localhost:3000/anything-else` → Error message

Expected Output Example:

```
{  
  "message": "Welcome to our Node.js JSON Server!"  
}
```

Module 5: Class Work

Task 1

Create a route `/courses` that returns:

```
[
  { "id": 1, "name": "HTML Basics" },
  { "id": 2, "name": "Node.js Fundamentals" },
  { "id": 3, "name": "React Introduction" }
]
```

 Task 2

Add a `/contact` route that returns your name, email, and phone number in JSON format.

 Module 6: Introducing Express.js

 What is Express.js?

Express.js is a **web framework** for Node.js that simplifies server creation and routing.

It allows you to:

- Build APIs faster
- Handle routes more easily
- Use **middleware** (special functions that handle requests before responses are sent)

 Why Do We Need Express.js?

Challenge with Plain Node.js	How Express Helps
You manually check <code>req.url</code> for every route	Express has built-in router (<code>app.get()</code> , <code>app.post()</code>)
You manually set headers and status codes	Express handles them automatically
Handling POST data is complex	Express provides body parsers
Difficult to organize large apps	Express supports modular route files
No easy middleware support	Express allows middleware for logging, auth, etc.

How Express Makes Things Easier

Install Express

```
npm install express
```

server.js (Express Version)

```
const express = require("express");
const app = express();

app.get("/", (req, res) => {
  res.json({ message: "Welcome to Express.js Server!" });
});

app.get("/students", (req, res) => {
  const students = [
    { id: 1, name: "John Doe", course: "Node.js" },
    { id: 2, name: "Jane Smith", course: "JavaScript" },
  ];
  res.json(students);
});

app.get("/info", (req, res) => {
  res.json({ appName: "Express JSON Server", version: "1.0.0" });
});

app.listen(3000, () => {
  console.log("Express server running on http://localhost:3000");
});
```

Why Express Is Better

-  Shorter and cleaner code
-  Automatic JSON handling
-  Easier routing
-  Middleware support (for logging, authentication, etc.)
-  More scalable for large applications



Module 7: Summary

Concept	Definition	Example
http module	Native Node.js server module	<code>require('http')</code>
<code>JSON.stringify()</code>	Converts data to JSON format	<code>JSON.stringify({name: 'Lee'})</code>
<code>res.setHeader()</code>	Sets content type	<code>'application/json'</code>
Express.js	A Node.js web framework	<code>npm install express</code>
<code>app.get()</code>	Handles GET requests	<code>app.get('/', ...)</code>
<code>res.json()</code>	Sends JSON response automatically	<code>res.json({ message: 'ok' })</code>



Class Discussion

1. Why do we use JSON in web applications?
2. What are the differences between the native Node.js server and Express.js?
3. How can Express make teamwork easier in large projects?